CampusOS
Complete ER & EER System Diagram

Current, detailed model of the website database: academic records, teaching, enrollment, attendance, accounts, and the additive Person identity layer.

18 tables | **18 foreign keys** | **4 ER / EER diagrams** | **SQL cookbook**

Read this model accurately. The existing `student` and `faculty` tables remain the authoritative academic records. The new `campus_person` supertype and role-link tables are optional and currently have no automatic matching/merge. Admins explicitly assert shared identity. Membership is partial and overlapping.

Implementation scope: this atlas documents the Next.js + SQLite implementation. The independent PostgreSQL + FastAPI build was not altered by the Person/weak-contact extension.

Inspired by the supplied Hospital Management DBMS report

The reference report separates the ER figure, entity classification, relationship/cardinality explanation, and implementation evidence. This edition follows that organization for CampusOS and includes the actual current EER extension rather than copying the hospital model.

Notation, keys & interpretation

Entity box One database table.	**PK** Primary key; underlined.	**FK** Foreign key to parent table.	**UQ*** Part of a composite UNIQUE tuple.
NULL Column is nullable.	**Arrow** Child FK points to parent.	**0..1 / 0..N** Optional one / many; check participation table.	**ISA** Only the explicit Person role links.

Cards list every column on their diagram page. Relationship paths label the child FK and referenced parent. The full relationship register gives min/max participation on both sides; the card arrows are a visual guide, not a replacement for that register.

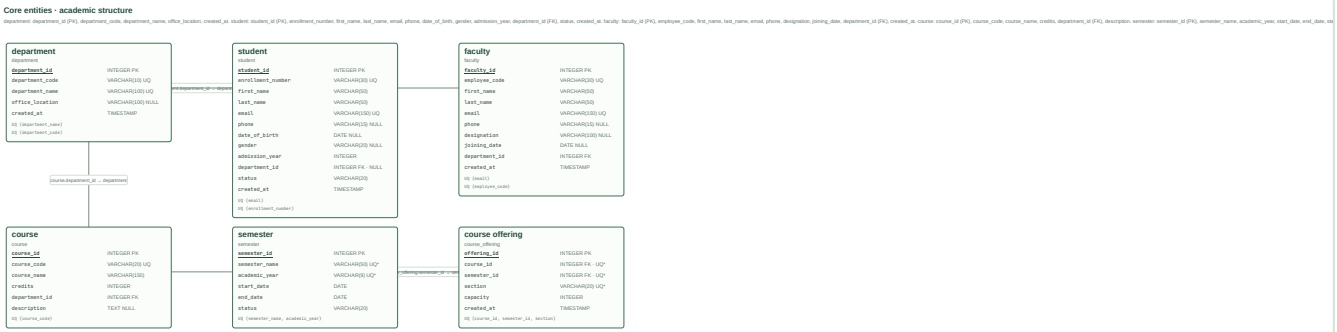
A NOT NULL foreign key means each existing child has one parent; it does **not** mean the parent must have a child. A composite unique pair is not a partial key unless it belongs to a weak entity's owner-relative key.

Entity-set categories

Category	Current CampusOS entities	Reason
Core academic	department, student, faculty, course, semester, course_offering	Organization, people-in-roles, catalog and scheduled terms.
Associative / transactional	faculty_assignment, enrollment	Record relationships/events and their own dates or statuses.
Dependent records	grade, attendance, person_contact, portal_session	Depend on a parent FK; person_contact additionally has owner-relative identity.
Operational/auth	portal_account, login_throttle, portal_migration	Security and implementation support, not academic domain concepts.
EER identity extension	campus_person, person_student, person_faculty, person_contact	Explicit optional Person identity with partial/overlapping subtype membership and weak contacts.

Core entities · academic structure

All attributes; solid FK paths are associations, never ISA inheritance.



PK = primary key · FK = foreign key · UQ* = composite unique tuple · NULL = nullable. Labels on paths identify the child column and referenced parent.

Academic transactions · teaching to assessment

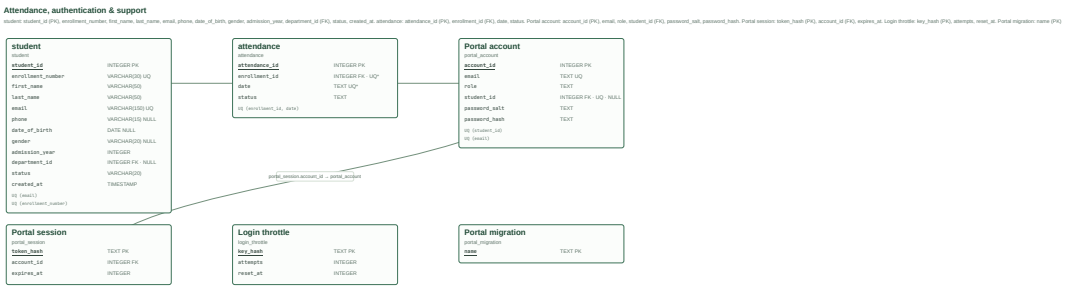
Associative entities resolve many-to-many links; grade is optional per enrollment.



PK = primary key · FK = foreign key · UQ* = composite unique tuple · NULL = nullable. Labels on paths identify the child column and referenced parent.

Attendance, authentication & support

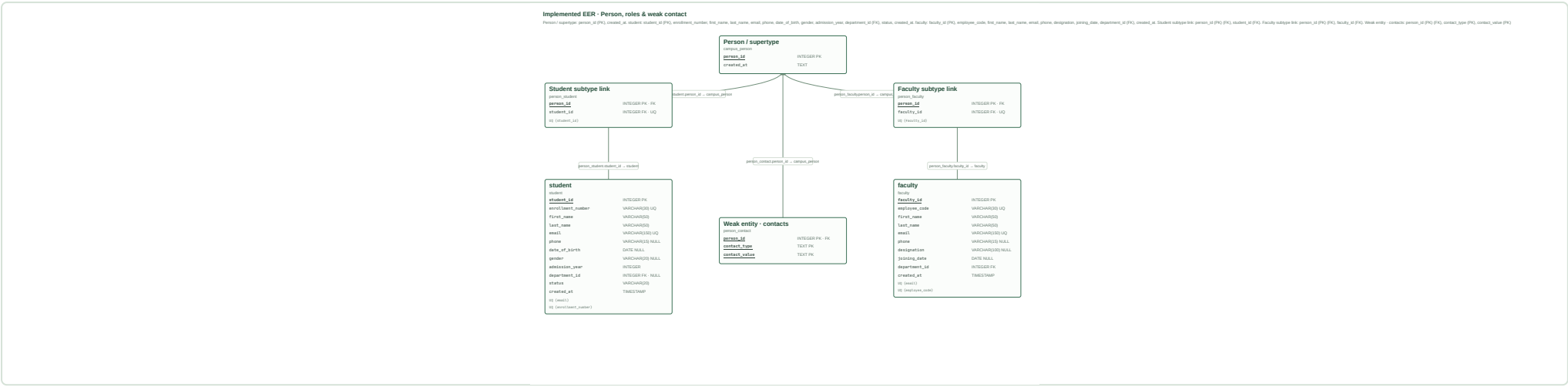
Student access remains read-only; authentication records are separate from academic identities.



PK = primary key · FK = foreign key · UQ* = composite unique tuple · NULL = nullable. Labels on paths identify the child column and referenced parent.

Implemented EER · Person, roles & weak contact

Partial + overlapping specialization; links are explicit and legacy IDs remain unchanged.



PK = primary key · FK = foreign key · UQ* = composite unique tuple · NULL = nullable. Labels on paths identify the child column and referenced parent.

Entity dictionary · academic people & departments

department

department

department_id	INTEGER	PK
department_code	VARCHAR(10)	UQ
department_name	VARCHAR(100)	UQ
office_location	VARCHAR(100)	NULL
created_at	TIMESTAMP	DEFAULT

UQ(department_name) · UQ(department_code)

Academic department; students may be unassigned.

student

student

student_id	INTEGER	PK
enrollment_number	VARCHAR(30)	UQ
first_name	VARCHAR(50)	
last_name	VARCHAR(50)	
email	VARCHAR(150)	UQ
phone	VARCHAR(15)	NULL
date_of_birth	DATE	NULL
gender	VARCHAR(20)	NULL
admission_year	INTEGER	
department_id	INTEGER	FK NULL
status	VARCHAR(20)	DEFAULT
created_at	TIMESTAMP	DEFAULT

UQ(email) · UQ(enrollment_number)

Legacy academic identity; unchanged by EER extension.

faculty

faculty

faculty_id	INTEGER	PK
employee_code	VARCHAR(30)	UQ
first_name	VARCHAR(50)	
last_name	VARCHAR(50)	
email	VARCHAR(150)	UQ
phone	VARCHAR(15)	NULL
designation	VARCHAR(100)	NULL
joining_date	DATE	NULL
department_id	INTEGER	FK
created_at	TIMESTAMP	DEFAULT

UQ(email) · UQ(employee_code)

Legacy employee identity; unchanged by EER extension.

Entity dictionary · catalog & terms

course

`course`

course_id	INTEGER	PK
course_code	VARCHAR(20)	UQ
course_name	VARCHAR(150)	
credits	INTEGER	
department_id	INTEGER	FK
description	TEXT	NULL

`UQ(course_code)`

Reusable catalogue course.

semester

`semester`

semester_id	INTEGER	PK
semester_name	VARCHAR(50)	UQ*
academic_year	VARCHAR(9)	UQ*
start_date	DATE	
end_date	DATE	
status	VARCHAR(20)	DEFAULT

`UQ(semester_name, academic_year)`

Academic term.

course offering

`course_offering`

offering_id	INTEGER	PK
course_id	INTEGER	FK UQ*
semester_id	INTEGER	FK UQ*
section	VARCHAR(20)	UQ*
capacity	INTEGER	
created_at	TIMESTAMP	DEFAULT

`UQ(course_id, semester_id, section)`

Course + semester + section.

Entity dictionary · teaching & assessment

faculty assignment
`faculty_assignment`

assignment_id	INTEGER	PK
faculty_id	INTEGER	FK UQ*
offering_id	INTEGER	FK UQ*
assigned_at	TIMESTAMP	DEFAULT

`UQ(faculty_id, offering_id)`

Faculty ↔ offering associative entity.

enrollment
`enrollment`

enrollment_id	INTEGER	PK
student_id	INTEGER	FK UQ*
offering_id	INTEGER	FK UQ*
enrollment_date	DATE	DEFAULT
enrollment_status	VARCHAR(20)	DEFAULT

`UQ(student_id, offering_id)`

Student ↔ offering associative entity.

grade
`grade`

grade_id	INTEGER	PK
enrollment_id	INTEGER	FK UQ
marks_obtained	REAL	NULL
max_marks	REAL	
grade_letter	VARCHAR(2)	NULL
updated_at	TIMESTAMP	DEFAULT

`UQ(enrollment_id)`

Optional one-to-one assessment for an enrollment.

Entity dictionary · attendance

attendance

attendance

attendance_id	INTEGER	PK
enrollment_id	INTEGER	FK UQ*
date	TEXT	UQ*
status	TEXT	

UQ(enrollment_id, date)

Attendance entry by enrollment and date.

Entity dictionary · implemented EER layer

Person / supertype
`campus_person`

person_id	INTEGER	PK
created_at	TEXT	DEFAULT

Optional shared identity; roles are explicitly linked, never inferred.

Student subtype link
`person_student`

person_id	INTEGER	PK FK
student_id	INTEGER	FK UQ

`UQ(student_id)`

Person role link to one legacy student; each student linked at most once.

Faculty subtype link
`person_faculty`

person_id	INTEGER	PK FK
faculty_id	INTEGER	FK UQ

`UQ(faculty_id)`

Person role link to one legacy faculty; each faculty linked at most once.

Weak entity · contacts
`person_contact`

person_id	INTEGER	PK FK
contact_type	TEXT	PK
contact_value	TEXT	PK

Weak contact. Key: owner + type + value; contact type/value is a partial key.

Entity dictionary · authentication & support

Portal account
`portal_account`

account_id	INTEGER	PK
email	TEXT	UQ
role	TEXT	
student_id	INTEGER	FK UQ NULL
password_salt	TEXT	
password_hash	TEXT	

`UQ(student_id) · UQ(email)`

Authentication role, isolated from Person ISA.

Portal session
`portal_session`

token_hash	TEXT	PK
account_id	INTEGER	FK
expires_at	INTEGER	

Session with hashed token and account FK.

Login throttle
`login_throttle`

key_hash	TEXT	PK
attempts	INTEGER	
reset_at	INTEGER	

Security state keyed by hashed email identity.

Portal migration
`portal_migration`

name	TEXT	PK

One-time seed migration marker.

Relationships, cardinality & participation

Parent → child. “Children per parent” describes the parent’s participation; “parents per child” describes each child’s allowed parent count.

Relationship	Enforced FK	Children / parent	Parents / child	Delete / update
department → student	`department_id → department.department_id`	0..N	0..1	RESTRICT / CASCADE
department → faculty	`department_id → department.department_id`	0..N	1..1	RESTRICT / CASCADE
department → course	`department_id → department.department_id`	0..N	1..1	RESTRICT / CASCADE
semester → course_offering	`semester_id → semester.semester_id`	0..N	1..1	RESTRICT / CASCADE
course → course_offering	`course_id → course.course_id`	0..N	1..1	RESTRICT / CASCADE
course_offering → faculty_assignment	`offering_id → course_offering.offering_id`	0..N	1..1	RESTRICT / CASCADE
faculty → faculty_assignment	`faculty_id → faculty.faculty_id`	0..N	1..1	RESTRICT / CASCADE
course_offering → enrollment	`offering_id → course_offering.offering_id`	0..N	1..1	RESTRICT / CASCADE
student → enrollment	`student_id → student.student_id`	0..N	1..1	RESTRICT / CASCADE
enrollment → grade	`enrollment_id → enrollment.enrollment_id`	0..1	1..1	RESTRICT / CASCADE
student → portal_account	`student_id → student.student_id`	0..1	0..1	RESTRICT / NO ACTION
portal_account → portal_session	`account_id → portal_account.account_id`	0..N	1..1	CASCADE / NO ACTION
student → person_student	`student_id → student.student_id`	0..1	1..1	RESTRICT / NO ACTION
campus_person → person_student	`person_id → campus_person.person_id`	0..1	1..1	CASCADE / NO ACTION
faculty → person_faculty	`faculty_id → faculty.faculty_id`	0..1	1..1	RESTRICT / NO ACTION
campus_person → person_faculty	`person_id → campus_person.person_id`	0..1	1..1	CASCADE / NO ACTION
campus_person → person_contact	`person_id → campus_person.person_id`	0..1	1..1	CASCADE / NO ACTION
enrollment → attendance	`enrollment_id → enrollment.enrollment_id`	0..N	1..1	RESTRICT / NO ACTION

Person ISA & partial-key relationship detail

Generalization and specialization in the implementation

`campus_person` is the common supertype key. `person_student(person_id PK/FK, student_id UQ/FK)` and `person_faculty(person_id PK/FK, faculty_id UQ/FK)` are subtype-link relations to existing student/faculty rows. The shared Person is explicitly created by an administrator. There is no inference by matching names or emails.

Specialization describes Person → Student / Faculty (top-down). Generalization describes seeing common identity across Student and Faculty and creating Person (bottom-up). Both describe this same ISA structure.

Weak entity and partial key

`person_contact` has full primary key `(person_id, contact_type, contact_value)`. Its partial key within the owner is `(contact_type, contact_value)`; the same type/value may occur for different Persons. The owner key is needed to identify the whole weak-entity row.

This is distinct from `enrollment` and `faculty_assignment`, which are associative entities with surrogate primary keys plus alternate pair uniqueness—not classic partial-key weak entities.

Admin usage: use the sidebar's "EER Identity" screen at `/eer`. Create a Person with no, one or both roles; choose existing student/faculty records to link; add or remove phone/email contacts. Unlinking preserves the source record.

No auto-merge, no new Person login and no change to student portal authorization. Person identity is an optional supplemental model. The separate PostgreSQL/FastAPI build is unaffected.

Admin panel: where names and records come from

Student list: actual SQL query template

```
SELECT s.*,
       s.first_name || ' ' || s.last_name AS full_name,
       d.department_name,
       (SELECT count(*) FROM enrollment
         WHERE student_id=s.student_id) AS enrollment_count
FROM student s
LEFT JOIN department d USING(department_id);
```

This template is `queries.students` in `web/lib/queries.mjs`. The Students screen uses `ResourcePage` with endpoint `/students`. API code applies requested filters, search, sort and pagination after fetching these query results.

Search a student's name directly

```
SELECT student_id, enrollment_number,
       first_name || ' ' || last_name AS full_name,
       email, status
FROM student
WHERE first_name LIKE '%' || ? || '%'
   OR last_name  LIKE '%' || ? || '%'
ORDER BY last_name, first_name;
```

This is a useful standalone SQL search example, **not** the precise implementation of the admin's search (the current API filters its joined result rows in JavaScript).

Faculty names and department

```
SELECT f.faculty_id, f.employee_code,
       f.first_name || ' ' || f.last_name AS full_name,
       f.email, d.department_name,
       f.designation
FROM faculty f
JOIN department d USING(department_id)
ORDER BY f.last_name, f.first_name;
```

Enrollment name + course + grade

```
SELECT st.enrollment_number,
       st.first_name || ' ' || st.last_name AS student_name,
       c.course_code, c.course_name,
       sem.semester_name, sem.academic_year,
       g.marks_obtained, g.max_marks, g.grade_letter
FROM enrollment e
JOIN student st USING(student_id)
JOIN course_offering o USING(offering_id)
JOIN course c USING(course_id)
JOIN semester sem USING(semester_id)
LEFT JOIN grade g USING(enrollment_id)
ORDER BY st.last_name, st.first_name;
```

Admin SQL examples · reports, EER, insert/update

Current report calculations

```
-- Students grouped by department
SELECT d.department_code, d.department_name,
       COUNT(s.student_id) AS students
FROM department d LEFT JOIN student s USING(department_id)
GROUP BY d.department_id
ORDER BY d.department_code;

-- Grade percentage (actual grades query calculation)
SELECT g.grade_id, st.first_name || ' ' || st.last_name AS student_name,
       c.course_code,
       ROUND(g.marks_obtained * 100.0 / g.max_marks, 2) AS percentage
FROM grade g JOIN enrollment e USING(enrollment_id)
JOIN student st USING(student_id)
JOIN course_offering o USING(offering_id)
JOIN course c USING(course_id);
```

Admin Reports screen calls `/reports/students`, `/reports/faculty`, or `/reports/academic`. Report calculations live in `web/lib/reports.mjs`; not all are a single database SQL statement.

Find EER linked names

```
SELECT person_id, student_name, faculty_name,
       student_enrollment_number, faculty_employee_code,
       contact_count
FROM v_person_eer
ORDER BY person_id;

SELECT pc.person_id, pc.contact_type,
       pc.contact_value
FROM person_contact pc
WHERE pc.person_id = ?;
```

CRUD statements represented by the API

```
-- Read one row by its table primary key
SELECT * FROM student WHERE student_id = ?;

-- Create / update examples (API binds parameters)
INSERT INTO student (enrollment_number, first_name, last_name, email,
                     admission_year, status)
VALUES (?, ?, ?, ?, ?, ?);

UPDATE student SET last_name = ? WHERE student_id = ?;

-- Delete is restricted by dependent foreign keys
DELETE FROM student WHERE student_id = ?;
```

The generic admin endpoint builds CRUD statements from a fixed resource/field allowlist in `web/lib/api.mjs`; values are bound separately. The browser does not send raw SQL.

How to see real executed templates

Admin → **SQL activity** polls `/dbviewer`. It shows redacted SQL templates, duration, route and row counts; it intentionally excludes credentials and does not log parameter values. It is bounded in-memory audit history, not a SQL editor.

> These are SQLite examples for the Next.js application. Do not paste SQLite syntax blindly into the separate PostgreSQL build.